

Министерство образования Республики Беларусь
Учреждение образования
«Белорусский государственный университет информатики
и радиоэлектроники»

Специальность «Программная инженерия»

Кафедра инженерной психологии и эргономики

Учебная дисциплина «Метрология, стандартизация и сертификация (в
информационных технологиях)»

Отчет

по лабораторной работе №2
«Метрики сложности потока управления программ»

Подготовил:

Студент гр. 410902
Большакова М.В.
Барбашёва А.А.

Проверил:

Болтак С. В.

Минск 2026

1 ЦЕЛЬ РАБОТЫ

Изучить метрики сложности программ на примере метрики Джилба.

2 ЗАДАНИЕ

Написать программу-парсер. Парсер может быть реализован на любом языке программирования и должен иметь графический интерфейс. Для анализа взять код на заданном языке. На основе анализа программа должна рассчитать метрику сложности Джилба: абсолютную сложность программы CL, относительную сложность программы cl и максимальный уровень вложенности управляющих конструкций CLI. Результат работы программы должен отображаться в наглядном виде на экране.

В данной работе представлен расчёт метрики Джилба для той же программы на языке C++, которая использовалась в лабораторной работе №1. Программа-парсер написана на языке Python с графическим интерфейсом на базе библиотеки tkinter.

Используются следующие показатели:

CL (абсолютная сложность программы) — суммарное число точек ветвления в программе. Точкой ветвления считается каждая конструкция if (включая каждый else if в цепочке), каждая «эффективная» ветвь оператора switch (число меток case/default минус одна) и каждый тернарный оператор «?:».

cl (относительная сложность программы) — отношение CL к общему числу операторов программы N: $cl = CL / N$. Показывает долю «разветвляющихся» операторов среди всех операторов программы.

CLI (индекс уровня вложенности) — максимальная глубина вложенности конструкций if/switch друг в друга (глубина, на которую увеличивается уровень при входе в очередную вложенную конструкцию ветвления).

Код анализируемой программы, написанный на языке C++ (тот же файл, что и в лабораторной работе №1):

```
import re
import tkinter as tk
from tkinter import ttk, filedialog, messagebox, scrolledtext

_COMMENT_OR_LITERAL = re.compile(
    r'//.*?$'
    r'|/\*.*?\*/'
    r'|"(?:\\.|[^\"])*"'
    r'|'(?:\\.|'[^']*')*",
    re.DOTALL | re.MULTILINE,
)

def strip_comments_and_literals(src: str) -> str:
    def _repl(m):
        s = m.group(0)
        if s.startswith('//') or s.startswith('/*'):
            return ''
        if s.startswith('"'):
            return ''
        return s
    return _COMMENT_OR_LITERAL.sub(_repl, src)

_TOKEN_RE = re.compile(r'[A-Za-z_]w*[\{\}\(\);:;,\?]\S')

def tokenize(src: str):
    return _TOKEN_RE.findall(src)

_LOOP_KEYWORDS = {'for', 'while', 'do'}
_STMT_START_KEYWORDS = {
    'if', 'else', 'for', 'while', 'do', 'switch', 'case', 'default',
    'return', 'break', 'continue', 'goto',
}
_DECL_KEYWORDS = {'enum', 'struct', 'class', 'union', 'typedef'}

class GilbParser:
    def __init__(self, tokens):
        self.toks = tokens
        self.n = len(tokens)
        self.pos = 0

        self.CL = 0
        self.max_depth = 0
        self.total_operators = 0

    def peek(self, offset=0):
        i = self.pos + offset
        return self.toks[i] if 0 <= i < self.n else None
```

```

def advance(self):
    t = self.toks[self.pos]
    self.pos += 1
    return t

def skip_balanced(self, open_c, close_c):
    depth = 0
    while self.pos < self.n:
        t = self.advance()
        if t == open_c:
            depth += 1
        elif t == close_c:
            depth -= 1
            if depth == 0:
                return

def parse_program(self):
    while self.pos < self.n:
        self.parse_statement(depth=0)

def parse_type_declaration(self, depth):
    """Пропускает объявление enum/struct/class/union/typedef,
    не считая его оператором."""
    # пропускаем само ключевое слово
    self.advance()

    # пропускаем имя типа и всё до '{' или ';'
    while self.pos < self.n and self.peek() not in ('{', ';', None):
        self.advance()

    if self.peek() == '{':
        # пропускаем тело { ... }
        self.skip_balanced('{', '}')
        # после тела может быть ';' или имя переменной
        while self.pos < self.n and self.peek() not in (';', None):
            self.advance()
        if self.peek() == ';':
            self.advance()
        elif self.peek() == ':':
            self.advance()
    elif self.peek() == ':':
        self.advance()

def parse_statement(self, depth):
    t = self.peek()
    if t is None:
        return

    nxt = self.peek(1)
    if (
        t not in _STMT_START_KEYWORDS
        and re.match(r'[A-Za-z_]\w*$', t)
        and nxt == ':'
        and self.peek(2) != ':'
    ):
        self.advance()
        self.advance()
        self.parse_statement(depth)

```

```

        return

    if t == '{':
        self.parse_compound(depth)
    elif t == 'if':
        self.parse_if(depth)
    elif t in _LOOP_KEYWORDS:
        self.parse_loop(depth, t)
    elif t == 'switch':
        self.parse_switch(depth)
    elif t in ('case', 'default'):
        self.parse_case_label(depth)
    elif t == 'try':
        self.advance()
        self.parse_body(depth)
        if self.peek() == 'catch':
            self.advance()
            if self.peek() == '(':
                self.skip_balanced('(', ')')
                self.parse_body(depth)
        return
    elif t in _DECL_KEYWORDS: # ← HOБАЯ BETKA
        self.parse_type_declaration(depth)
    else:
        self.parse_simple(depth)

def parse_compound(self, depth):
    self.advance()
    while self.peek() is not None and self.peek() != '}':
        self.parse_statement(depth)
    if self.peek() == '}':
        self.advance()

def parse_body(self, depth):
    if self.peek() == '{':
        self.parse_compound(depth)
    elif self.peek() == ';':
        self.advance()
    else:
        self.parse_statement(depth)

def parse_if(self, depth):
    self.CL += 1
    self.max_depth = max(self.max_depth, depth + 1)

    self.advance()
    if self.peek() == '(':
        self.skip_balanced('(', ')')

    self.parse_body(depth + 1)

    if self.peek() == 'else':
        self.advance()
        self.parse_body(depth + 1)

def count_switch_branches(self):
    if self.peek() != '{':

```

```

        return 0
    depth = 0
    count = 0
    i = self.pos
    while i < self.n:
        t = self.toks[i]
        if t == '{':
            depth += 1
        elif t == '}':
            depth -= 1
            if depth == 0:
                break
        elif depth == 1 and t in ('case', 'default'):
            count += 1
        i += 1
    return count

def parse_switch(self, depth):
    self.advance()
    if self.peak() == '(':
        self.skip_balanced('(', ')')

    branches = self.count_switch_branches()
    if branches >= 1:
        equiv_cl = max(branches - 1, 0)
        self.CL += equiv_cl
    equiv_depth = depth + 1 + max(branches - 2, 0)
    self.max_depth = max(self.max_depth, equiv_depth)

    if self.peak() == '{':
        self.parse_compound(depth + 1)
    else:
        self.parse_body(depth + 1)

def parse_case_label(self, depth):
    self.advance()
    while self.peak() not in (':', None):
        self.advance()
    if self.peak() == ':':
        self.advance()

def parse_loop(self, depth, kind):
    self.advance()

    if kind in ('for', 'while'):
        if self.peak() == '(':
            self.skip_balanced('(', ')')
            self.parse_body(depth)
        else:
            self.parse_body(depth)
            if self.peak() == 'while':
                self.advance()
                if self.peak() == '(':
                    self.skip_balanced('(', ')')
                if self.peak() == ';':
                    self.advance()

```

```

def parse_simple(self, depth):
    self.total_operators += 1
    paren_depth = 0
    while self.pos < self.n:
        t = self.advance()
        if t == '?':
            self.CL += 1
            self.max_depth = max(self.max_depth, depth)
        elif t == '(':
            paren_depth += 1
        elif t == ')':
            paren_depth -= 1
        elif t == ';' and paren_depth <= 0:
            break
        elif t == '{' and paren_depth <= 0:
            self.pos -= 1
            self.parse_compound(depth)
            break

```

```

def analyze_source(src: str) -> dict:
    clean = strip_comments_and_literals(src)
    clean = re.sub(r'^\s*#.*$', '', clean, flags=re.MULTILINE)
    clean = re.sub(r'\busing\s+namespace\b(?:;)*;', '', clean)
    tokens = tokenize(clean)
    parser = GilbParser(tokens)
    parser.parse_program()

```

```

CL = parser.CL
total = parser.total_operators
cl = CL / total if total else 0.0
CLI = parser.max_depth

```

```

return {
    'CL': CL,
    'cl': round(cl, 4),
    'CLI': CLI,
    'total_operators': total,
}

```

----- GUI -----

```

class GilbApp:
    def __init__(self, root):
        self.root = root
        self.root.title("Анализ сложности C++ программы (метрика Гилба)")
        self.root.geometry("950x700")
        self.root.minsize(750, 550)

        self._build_ui()

    def _build_ui(self):
        # Верхняя панель с кнопками
        top_frame = ttk.Frame(self.root, padding=8)
        top_frame.pack(fill=tk.X)

```

```

ttk.Button(top_frame, text="Открыть .cpp файл",
            command=self.open_file).pack(side=tk.LEFT, padx=(0, 6))
ttk.Button(top_frame, text="Очистить",
            command=self.clear_all).pack(side=tk.LEFT, padx=(0, 6))
ttk.Button(top_frame, text="Анализировать",
            command=self.analyze).pack(side=tk.LEFT, padx=(0, 6))

self.file_label = ttk.Label(top_frame, text="Файл не выбран",
                             foreground="gray")
self.file_label.pack(side=tk.LEFT, padx=12)

# Метка и поле с исходным кодом
ttk.Label(self.root, text="Исходный код C++:",
           padding=(8, 4, 8, 0)).pack(anchor=tk.W)

code_frame = ttk.Frame(self.root, padding=(8, 0, 8, 8))
code_frame.pack(fill=tk.BOTH, expand=True)

self.code_text = scrolledtext.ScrolledText(
    code_frame, wrap=tk.NONE, font=("Consolas", 10),
    undo=True
)
self.code_text.pack(fill=tk.BOTH, expand=True)

# Панель результатов
result_frame = ttk.LabelFrame(self.root, text="Результат работы парсера",
                               padding=10)
result_frame.pack(fill=tk.X, padx=8, pady=(0, 10))

self.cl_var = tk.StringVar(value="—")
self.cl_rel_var = tk.StringVar(value="—")
self.cli_var = tk.StringVar(value="—")
self.total_var = tk.StringVar(value="—")

def add_row(row, label, var, color="#1a4d8f"):
    ttk.Label(result_frame, text=label,
              font=("Segoe UI", 10)).grid(
        row=row, column=0, sticky=tk.W, padx=(0, 12), pady=3)
    ttk.Label(result_frame, textvariable=var,
              font=("Segoe UI", 11, "bold"),
              foreground=color).grid(
        row=row, column=1, sticky=tk.W, pady=3)

add_row(0, "Абсолютная сложность программы (CL):",
        self.cl_var, "#1a4d8f")
add_row(1, "Относительная сложность программы (cl):",
        self.cl_rel_var, "#1a4d8f")
add_row(2, "Максимальный уровень вложенности (CLI):",
        self.cli_var, "#8f1a1a")
add_row(3, "Общее число операторов программы:",
        self.total_var, "#555555")

# ----- действия -----

def open_file(self):
    path = filedialog.askopenfilename(
        title="Выберите .cpp файл",

```



```

        filetypes=[("C++ файлы", "*.cpp *.cc *.cxx *.c++ *.h *.hpp"),
                    ("Все файлы", ".*")]
    )
    if not path:
        return
    try:
        with open(path, encoding='utf-8') as f:
            content = f.read()
    except UnicodeDecodeError:
        try:
            with open(path, encoding='cp1251') as f:
                content = f.read()
        except Exception as e:
            messagebox.showerror("Ошибка",
                                f"Не удалось прочитать файл:\n{e}")
            return
    except Exception as e:
        messagebox.showerror("Ошибка",
                                f"Не удалось открыть файл:\n{e}")
        return

    self.code_text.delete("1.0", tk.END)
    self.code_text.insert("1.0", content)
    self.file_label.config(text=path, foreground="black")

def clear_all(self):
    self.code_text.delete("1.0", tk.END)
    self.file_label.config(text="Файл не выбран", foreground="gray")
    for var in (self.cl_var, self.cl_rel_var,
                self.cli_var, self.total_var):
        var.set("")

def analyze(self):
    src = self.code_text.get("1.0", tk.END)
    if not src.strip():
        messagebox.showwarning("Внимание",
                                "Введите код или откройте файл.")
        return
    try:
        result = analyze_source(src)
    except Exception as e:
        messagebox.showerror("Ошибка анализа",
                                f"Не удалось выполнить анализ:\n{e}")
        return

    self.cl_var.set(str(result['CL']))
    self.cl_rel_var.set(str(result['cl']))
    self.cli_var.set(str(result['CLI']))
    self.total_var.set(str(result['total_operators']))

def main():
    root = tk.Tk()
    try:
        style = ttk.Style()
        if "vista" in style.theme_names():
            style.theme_use("vista")
    
```

```

        elif "clam" in style.theme_names():
            style.theme_use("clam")
except Exception:
    pass

```

```

GilbApp(root)
root.mainloop()

```

```

if __name__ == '__main__':
    main()

```

3 РЕЗУЛЬТАТЫ РАБОТЫ ПРОГРАММЫ

Программа-парсер написана на языке Python с графическим интерфейсом на базе библиотеки tkinter. Окно программы отображает исходный код анализируемой программы и итоговые значения метрики Джилба. Результат работы программы представлен на рисунке 1.

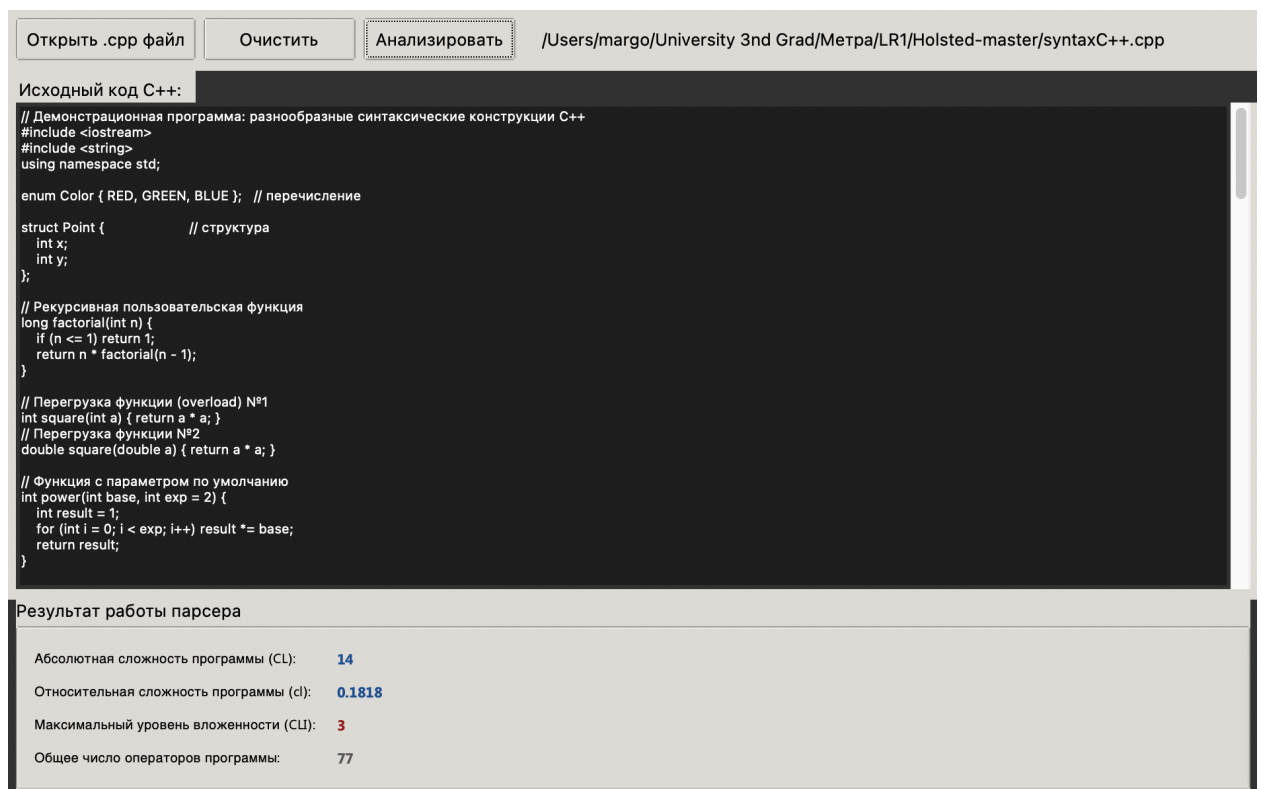


Рисунок 1 — Окно программы: исходный код и результат расчёта метрики Джилба

4 РЕЗУЛЬТАТ РАСЧЕТА МЕТРИК

Схема алгоритма представлена на изображении СхемаАлгоритм-Джилб.png в этом же репозитории.

Суммарная абсолютная сложность: $CL = 1+1+1+1+1+3+1+1+1+1+1+1 = 14$, что совпадает со значением, рассчитанным программой.

Максимальный уровень вложенности достигается в цепочке if / else if / else if программы main() (строки 2–4 таблицы): $CLI = 3$, что также совпадает с результатом работы программы.

Общее число операторов программы (простых операторов, отделяемых «;») составило $N = 77$, следовательно относительная сложность:

$$cl = CL / N = 14 / 77 \approx 0,1818.$$

Все три значения, рассчитанные вручную ($CL = 14$, $CLI = 3$, $cl \approx 0,1818$), совпали со значениями, выведенными программой на рисунке 1, что подтверждает корректность работы парсера.

Программа-парсер написана на языке Python с использованием стандартной библиотеки tkinter (модуль ttk) для графического интерфейса. Лексический и синтаксический разбор кода реализован без внешних библиотек: исходный текст программы предварительно очищается от комментариев, строковых литералов и директив препроцессора, после чего производится рекурсивный разбор управляющих конструкций (if/else, switch/case, циклы for/while/do...while, try/catch) с подсчётом точек ветвления и глубины их вложенности. Результат работы программы представлен на рисунке 1.